МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Вычислительная математика»

Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342 Ларионов В.А.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Реализовать метод Гаусса, протестировать его на матрицах различного размера, сравнить точность с существующими реализациями методов решения систем линейных уравнений.

Теоретический минимум.

Метод Гаусса это классический алгоритм последовательного исключения неизвестных для решения систем линейных уравнений. Он приводит матрицу системы к треугольному виду с помощью элементарных преобразований, после чего переменные находятся обратной подстановкой. Метод универсален и подходит для систем с любым числом решений.

Существуют модификации алгоритма для повышения точности:

1) Метод частичного выбора - на каждом шаге выбирается максимальный элемент из столбца;
2) Метод полного выбора - на каждом шаге выбирается максимальный элемент из всех элементов по индексу больших текущего.

Выполнение работы.

В качестве модификации метода Гаусса был выбран метод частичного выбора, трудоемкость по операциям составляет $O(\frac{2m^3}{3})$ при достаточной для учебных задач точности. На каждом шаге метода частичного выбора выбирается максимальный элемент в столбце, после чего происходит перестановка строки текущего ведущего элемента с найденным.

При запуске написанной программы, создается и заполняется числами выбранного типа (int, float, double, long double) квадратная матрица A размером N, которое вводит пользователь. Сгенерированная матрица передается в функцию, которая генерирует вектор X, содержащий решение системы уравнений, перемножением матрицы A на вектор X получает вектор B и возвращает пару X, B. После, в зависимости от выбора пользователя, матрица A и вектор B передается в функцию, которая возвращает вектор решения $\overline{X}$. Далее вектор X и $\overline{X}$ сравниваются.

Одной из опций на этапе выполнения программы является сравнение реализации метода Гаусса методом частичного выбора и реализация из библиотеки Eigen (в ней используется другой метод). Результаты сравнения погрешности и скорости выполнения представлены на Рисунке 1 и Рисунке 2.

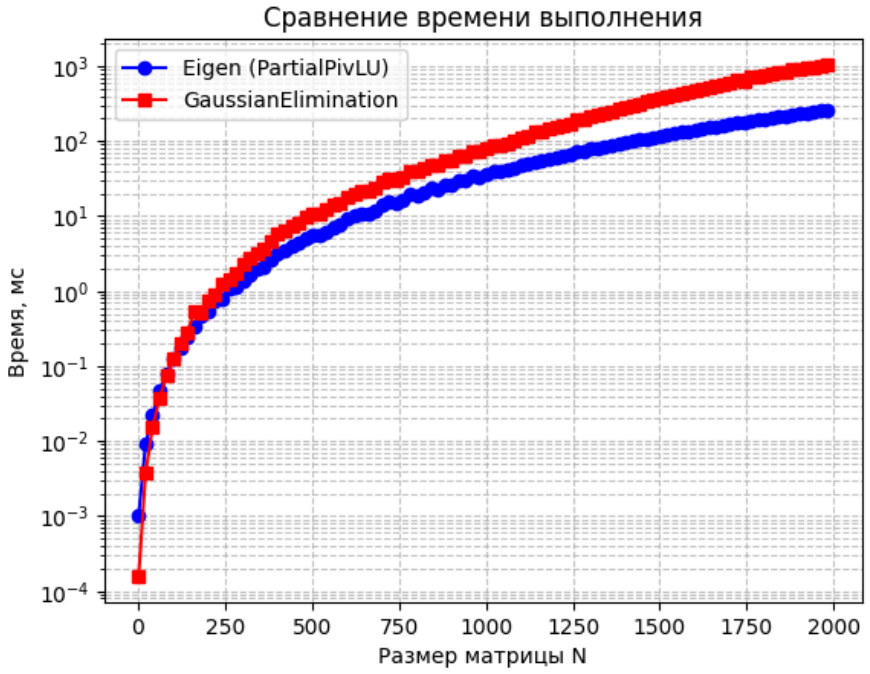

Рисунок 1 - Сравнение времени выполнения

Рисунок 2 - Сравнение погрешности

Выводы.

В ходе выполнения работы был реализован метод Гаусса с частичным выбором, проведено сравнение времени работы (быстродействия) и относительной погрешности реализованного метода и реализации из библиотеки Eigen.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include "Eigen/Dense"
#include "../utils/matrixlib.hpp"
#include <chrono>

#define REPEAT_COUNT 10

using type = float;

template <class T>
std::pair<std::vector<T>, std::vector<T>>
getCorrectResponseVectors(squareMatrix<T>& M) {
    int N = M.getN();
    std::vector<T> X(N, 0), B(N, 0);
    if (!M.isSingular()) {
        for (int i = 0; i < N; ++i) {
            X[i] = M.getRandomNumber();
        }
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                B[i] += (M.getElement(i, j) * X[j]);
            }
        }
    }
    return {X, B};
}

template <class T>
std::vector<T> GaussianElimination(squareMatrix<T>& M, std::vector<T>& B) {
    int N = M.getN();
    std::vector<T> X(N, 0);
    for (int i = 0; i < N - 1; ++i) {
        int RowOfMaxElementInColumn = M.getRowOfMaxElement(i, i);
        if (RowOfMaxElementInColumn != i && RowOfMaxElementInColumn != -1) {
            M.swapRows(RowOfMaxElementInColumn, i);
            std::swap(B[RowOfMaxElementInColumn], B[i]);
        }
        auto& row_i = M.getRow(i);
        for (int j = i + 1; j < N; ++j) {
            auto& row_j = M.getRow(j);
            T factor = row_j[i] / row_i[i];
            for (int k = i; k < N; ++k)
                row_j[k] -= factor * row_i[k];
            B[j] -= factor * B[i];
        }
    }
    for (int i = N - 1; i >= 0; --i) {
        const auto& row_i = M.getRow(i);
        T sum = 0;
        for (int j = i + 1; j < N; ++j) {
            sum += row_i[j] * X[j];
        }
        X[i] = (B[i] - sum) / row_i[i];
    }
    return X;
}

int getData() {
    std::fstream log("log.txt", std::ios::out);
```

```
    std::vector<std::vector<long double>> results;
    results.reserve(200);

    for (int i = 2; i < 2000; i += 20) {
        int count = 0;
        std::vector<long double> row(5);
        row[0] = i;
        while(count++ != REPEAT_COUNT) {
            squareMatrix<type> M_original(i);
            auto vectors = getCorrectResponseVectors<type>(M_original);

            squareMatrix<type> M_for_gaussian = M_original;

            Eigen::MatrixXf eigenM(i, i);
            Eigen::VectorXf eigenB(i);
            for (int k = 0; k < i; ++k) {
                for (int j = 0; j < i; ++j) {
                    eigenM(k, j) = M_original.getElement(k, j);
                }
                eigenB(k) = vectors.second[k];
            }

            auto startEigen = std::chrono::high_resolution_clock::now();
            Eigen::VectorXf eigenX = eigenM.partialPivLu().solve(eigenB);
            auto endEigen = std::chrono::high_resolution_clock::now();
                  row[1] += std::chrono::duration<double, std::milli>(endEigen -
startEigen).count();

            long double err_eigen = 0.0;
            int valid_count_eigen = 0;
            for (int k = 0; k < i; ++k) {
                if (std::fabs(vectors.first[k]) > 1e-10) {
                    double d = std::fabs(vectors.first[k] - eigenX(k));
                    err_eigen += d / std::fabs(vectors.first[k]);
                    valid_count_eigen++;
                }
            }
              row[2] += (valid_count_eigen > 0) ? err_eigen / valid_count_eigen :
0.0;

            auto start = std::chrono::high_resolution_clock::now();
              std::vector<type> answer = GaussianElimination<type>(M_for_gaussian,
vectors.second);
            auto end = std::chrono::high_resolution_clock::now();
                      row[3] += std::chrono::duration<double, std::milli>(end -
start).count();

            long double err_gaussian = 0.0;
            int valid_count_gaussian = 0;
            for (int k = 0; k < i; ++k) {
                if (std::fabs(vectors.first[k]) > 1e-10) {
                    double d = std::fabs(vectors.first[k] - answer[k]);
                    err_gaussian += d / std::fabs(vectors.first[k]);
                    valid_count_gaussian++;
                }
            }
                       row[4] += (valid_count_gaussian > 0) ? err_gaussian /
valid_count_gaussian : 0.0;
        }
        row[1] /= REPEAT_COUNT;
        row[2] /= REPEAT_COUNT;
        row[3] /= REPEAT_COUNT;
        row[4] /= REPEAT_COUNT;
```

```
        results.push_back(row);
    }
    for (const auto& row : results) {
        for (auto val : row) log << val << '\t';
        log << '\n';
    }
    return EXIT_SUCCESS;
}

int TestGaussian() {
    int N = 0;
    std::cin >> N;
    squareMatrix<type> M(N);
    auto vectors = getCorrectResponseVectors<type>(M);
    auto start = std::chrono::high_resolution_clock::now();
    std::vector<type> answer = GaussianElimination<type>(M, vectors.second);
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> duration = end - start;
    std::cout << "Время метода GaussianElimination: " << duration.count() << " мс"
<< std::endl;
    long double errorNorm = 0.0;
    for (int i = 0; i < N; ++i) {
                errorNorm  +=  (vectors.first[i]  -  answer[i])*(vectors.first[i]  -
answer[i]);
    }
     std::cout  <<  "||точный_X - GaussianElimination|| = " << std::sqrt(errorNorm)
<< std::endl;
    return EXIT_SUCCESS;
}

int CompareGaussianAndLU() {
    int N = 0;
    std::cin >> N;
    squareMatrix<type> M(N);
    auto vectors = getCorrectResponseVectors<type>(M);
    Eigen::MatrixXf eigenM(N, N);
    Eigen::VectorXf eigenB(N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            eigenM(i, j) = M.getElement(i, j);
        }
        eigenB(i) = vectors.second[i];
    }

    auto startEigen = std::chrono::high_resolution_clock::now();
    Eigen::VectorXf eigenX = eigenM.partialPivLu().solve(eigenB);
    auto endEigen = std::chrono::high_resolution_clock::now();
         std::chrono::duration<double,  std::milli>  durationEigen  =  endEigen  -
startEigen;
      std::cout  <<  "Время  решения  Eigen:  "  <<  durationEigen.count()  <<  " мс" <<
std::endl;

    auto start = std::chrono::high_resolution_clock::now();
    std::vector<type> answer = GaussianElimination<type>(M, vectors.second);
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> duration = end - start;
    std::cout << "Время метода GaussianElimination: " << duration.count() << " мс"
<< std::endl;

    long double diffNorm = 0.0;
    for (int i = 0; i < N; ++i) {
        diffNorm += (vectors.first[i] - eigenX(i))*(vectors.first[i] - eigenX(i));
    }
```

```
      std::cout << "||точный_X - eigen_решение|| = " << std::sqrt(diffNorm) << std::endl;

    long double errorNorm = 0.0;
    for (int i = 0; i < N; ++i) {
                errorNorm += (vectors.first[i] - answer[i])*(vectors.first[i] - answer[i]);
    }
     std::cout << "||точный_X - GaussianElimination|| = " << std::sqrt(errorNorm) << std::endl;
    return EXIT_SUCCESS;
}

int main() {
    std::cout << "Выберете из списка:\n";
     std::cout << "1 - Получить и вывести данные для построения графика сравнения времени выполнения и точности для методов\n";
       std::cout << "2 - Сравнить метод GaussianElimination с методом LU из библиотеки Eigen для матрицы N на N\n";
    std::cout << "3 - Выполнить GaussianElimination для матрицы N на N\n";
    int choice;
    std::cin >> choice;
    switch (choice)
    {
    case 1:
        return getData();
    case 2:
        return CompareGaussianAndLU();

    case 3:
        return TestGaussian();
    }
}
```

Файл matrixlib.hpp

```
/*
Библиотека для работы с матрицами.
*/
#include <limits>
#include <string>
#include <vector>
#include <iostream>
#include <fstream>
#include <iomanip>
#include <random>
#include <type_traits>

#define STANDART_MATRIX_SIZE 5
#define MAXIMUM_ELEMENT_VALUE 10
#ifndef EPSILON
#define EPSILON 1e-7
#endif
#ifndef GILBERT
```

```
#define GILBERT 0
#endif

template <class T>
class squareMatrix {
private:
    std::vector<std::vector<T>> M;
    int N;
    bool symmetric;
    bool singular;
private:
    bool checkSymmetric();
    bool checkSingular();
    bool isColumnsLinearlyDependent(int colI, int colJ);
public:
    squareMatrix(int matrixS);
    squareMatrix();
    friend std::ostream& operator<<(std::ostream& os, const squareMatrix& matrix)
{
        os << matrix.toStr();
        return os;
    }
    bool isSymmetric() const;
    bool isSingular() const;
    int getN() const;
    const T& getElement(int rowIndex, int columnIndex) const;
    void setElement(int rowIndex, int columnIndex, T value);
    std::string toStr() const;
    std::pair<int, int> getCoordOfMaxElement(int startPos) const;
    int getRowOfMaxElement(int colomn, int startRow) const;
    T getRandomNumber();
    T getGilbertRandom(int i, int j);
    void swapRows(int rowI, int rowJ);
    void swapCollumns(int colI, int colJ);
    T norm() const;
    squareMatrix<T> multiply(const squareMatrix<T>& other) const;
    squareMatrix<T> inverse() const;
    T conditionNumber() const;
    void multiplyRow(double long multiplier, int rowIndex, int startPos, int
endPos);
    void multiplyRow(double long multiplier, int rowIndex, int startPos);
    void multiplyRow(double long multiplier, int rowIndex);
```

```
    std::vector<T>& getRow(int index);
};


template <class T>
squareMatrix<T>::squareMatrix(int matrixSide) : N(matrixSide){
    M.resize(N, std::vector<T>(N));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (GILBERT) {
                M[i][j] = getGilbertRandom(i, j);
            }
            else {
                M[i][j] = getRandomNumber();
            }

        }
    }
    this->symmetric = false;
    this->singular = false;
}


template <class T>
squareMatrix<T>::squareMatrix() : squareMatrix(STANDART_MATRIX_SIZE) {}


template <class T>
bool squareMatrix<T>::isSymmetric() const {
    return this->symmetric;
}


template <class T>
bool squareMatrix<T>::isSingular() const {
    return this->singular;
}


template <class T>
int squareMatrix<T>::getN() const {
    return this->N;
```

```
}


template <class T>
T squareMatrix<T>::norm() const {
    T maxSum = 0;
    for (int j = 0; j < N; ++j) {
        T currentSum = 0;
        for (int i = 0; i < N; ++i) {
            currentSum += std::fabs(getElement(i, j));
        }
        if (currentSum > maxSum) {
            maxSum = currentSum;
        }
    }
    return maxSum;
}


template <class T>
squareMatrix<T> squareMatrix<T>::multiply(const squareMatrix<T>& other) const {
    squareMatrix<T> result(N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            T sum = 0;
            for (int k = 0; k < N; ++k) {
                sum += getElement(i, k) * other.getElement(k, j);
            }
            result.setElement(i, j, sum);
        }
    }
    return result;
}


template <class T>
squareMatrix<T> squareMatrix<T>::inverse() const {
    squareMatrix<T> A = *this;
    squareMatrix<T> I(N);
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            I.setElement(i, j, (i == j ? 1 : 0));
```

```
        }
    }
    for (int i = 0; i < N; ++i) {
        T pivot = A.getElement(i, i);
        for (int j = 0; j < N; ++j) {
            A.setElement(i, j, A.getElement(i, j) / pivot);
            I.setElement(i, j, I.getElement(i, j) / pivot);
        }
        for (int k = 0; k < N; ++k) {
            if (k == i) continue;
            T factor = A.getElement(k, i);
            for (int j = 0; j < N; ++j) {
                T valA = A.getElement(k, j) - factor * A.getElement(i, j);
                T valI = I.getElement(k, j) - factor * I.getElement(i, j);
                A.setElement(k, j, valA);
                I.setElement(k, j, valI);
            }
        }
    }
    return I;
}


template <class T>
T squareMatrix<T>::conditionNumber() const {
    if (this->isSingular()) {
        throw std::runtime_error("Matrix is singular");
    }
    squareMatrix<T> inv = this->inverse();
    T normA = this->norm();
    T normInv = inv.norm();
    return normA * normInv;
}


template <class T>
const T& squareMatrix<T>::getElement(int rowIndex, int columnIndex) const {
    return this->M[rowIndex][columnIndex];
}


template <class T>
```

```
void squareMatrix<T>::setElement(int rowIndex, int columnIndex, T value) {
    this->M[rowIndex][columnIndex] = value;
    return;
}


template <class T>
T squareMatrix<T>::getGilbertRandom(int i, int j) {
    return static_cast<T>(1.0/(i + j + 1));
}


template <class T>
T squareMatrix<T>::getRandomNumber() {
    if constexpr (std::is_same_v<T, int>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_int_distribution<int> dist(-MAXIMUM_ELEMENT_VALUE,
MAXIMUM_ELEMENT_VALUE);
        return dist(gen);
    }
    else if constexpr (std::is_same_v<T, double> || std::is_same_v<T, float> ||
                        std::is_same_v<T, long double>) {
        static std::random_device rd;
        static std::mt19937 gen(rd());
        static std::uniform_real_distribution<long double>
dist(-MAXIMUM_ELEMENT_VALUE, MAXIMUM_ELEMENT_VALUE);
        return static_cast<T>(dist(gen));
    }
    return static_cast<T>(0);
}


template <class T>
std::string squareMatrix<T>::toStr() const {
    std::string strMatrix;
    strMatrix.reserve(N * N * 15);
    for (const auto& row : M) {
        for (const auto&  element : row) {
            strMatrix += std::to_string(element);
            strMatrix += '\t';
        }
```

```
        strMatrix += '\n';
    }
    return strMatrix;
}


template <class T>
bool squareMatrix<T>::isColumnsLinearlyDependent(int colI, int colJ) {
    int indexFirstNonZeroCoefColI = -1;
    int indexFirstNonZeroCoefColJ = -1;
    for (int rowIndex = 0; rowIndex < this->N; ++rowIndex) {
        const T& currColIElement = getElement(rowIndex, colI);
        const T& currColJElement = getElement(rowIndex, colJ);
        if (indexFirstNonZeroCoefColI == -1 && currColIElement != 0) {
            indexFirstNonZeroCoefColI = rowIndex;
        }
        if (indexFirstNonZeroCoefColJ == -1 && currColJElement != 0) {
            indexFirstNonZeroCoefColJ = rowIndex;
        }
    }
    if (indexFirstNonZeroCoefColI == -1 || indexFirstNonZeroCoefColJ == -1) {
        return true;
    }
    if (indexFirstNonZeroCoefColI != indexFirstNonZeroCoefColJ) {
        return false;
    }
    const T& currColIElement = getElement(indexFirstNonZeroCoefColI, colI);
    const T& currColJElement = getElement(indexFirstNonZeroCoefColJ, colJ);
    const T coef = currColIElement / currColJElement;
    for (int rowIndex = 0; rowIndex < this->N; ++rowIndex) {
        const T& currColIElement2 = getElement(rowIndex, colI);
        const T& currColJElement2 = getElement(rowIndex, colJ);
        if (std::abs(currColJElement2 * coef - currColIElement2) > EPSILON) {
            return false;
        }
    }
    return true;
}


template <class T>
bool squareMatrix<T>::checkSingular() {
```

```
    if (GILBERT) {
        return false;
    }
    for (int indexColumnI = 0; indexColumnI < this->N; ++indexColumnI) {
        for (int indexColumnJ = indexColumnI + 1; indexColumnJ < this->N;
++indexColumnJ) {
            if (isColumnsLinearlyDependent(indexColumnI, indexColumnJ)) {
                return true;
            }
        }
    }
    return false;
}


template <class T>
bool squareMatrix<T>::checkSymmetric() {
    for(int i = 0; i < this->N; i++) {
        for(int j = i + 1; j < this->N; j++) {
            if (this->M[i][j] != this->M[j][i]) {
                return false;
            }
        }
    }
    return true;
}


template <class T>
std::pair<int, int> squareMatrix<T>::getCoordOfMaxElement(int startPos) const {
    std::pair<int, int> coord = {-1,-1};
    T maxElem = std::numeric_limits<T>::min();
    for(int i = startPos; i < N; ++i) {
        for(int j = startPos; j < N; ++j) {
            const T& currentElement = M[i][j];
            if (currentElement < EPSILON) {
                continue;
            }
            if (std::fabs(currentElement) > maxElem) {
                coord = {i, j};
                maxElem = std::fabs(currentElement);
            }
```

```
        }
    }
    return coord;
}


template <class T>
int squareMatrix<T>::getRowOfMaxElement(int colomn, int startRow) const {
    int targetRow = -1;
    T maxElem = std::numeric_limits<T>::min();
    for(int i = startRow; i < N; ++i) {
        const T& currentElement = std::fabs(M[i][colomn]);
        if (currentElement < EPSILON) {
            continue;
        }
        if (currentElement > maxElem) {
            targetRow = i;
            maxElem = currentElement;
        }
    }
    return targetRow;
}


template <class T>
void squareMatrix<T>::swapRows(int rowI, int rowJ) {
    if(rowI < N && rowJ < N && rowI >= 0 && rowJ >= 0) {
        std::swap(M[rowI], M[rowJ]);
    }
}


template <class T>
void squareMatrix<T>::multiplyRow(double long multiplier, int rowIndex, int
startPos, int endPos) {
    for (int i = startPos; i < endPos; ++i) {
        M[rowIndex][i] *= multiplier;
    }
}


template <class T>
```

```
void squareMatrix<T>::multiplyRow(double long multiplier, int rowIndex, int
startPos) {
    for (int i = startPos; i < N; ++i) {
        M[rowIndex][i] *= multiplier;
    }
}


template <class T>
void squareMatrix<T>::multiplyRow(double long multiplier, int rowIndex) {
    for (int i = 0; i < N; ++i) {
        M[rowIndex][i] *= multiplier;
    }
}


template <class T>
void squareMatrix<T>::swapCollumns(int colI, int colJ) {
    if(colI < N && colJ < N && colI >= 0 && colJ >= 0) {
        for (int i = 0; i < N; ++i) {
            std::swap(M[i][colI], M[i][colJ]);
        }
    }
}

template <class T>
std::vector<T>& squareMatrix<T>::getRow(int index) {
    return this->M[index];
}
```